

I SIFT

I.1 Local invariant features

Informative regions: which points carry more information.

Local invariant features: zones in the image which are numerically similar.

1. **Detection:** identify interest points in both images. *Harris corner, scale invariance, Censure.*
2. **Description:** Extract vector feature descriptor (make a vector with every point surrounding your interest points). *SIFT, ORB.*
3. **Matching:** Determine correspondence between descriptors in two views. The more alike those vectors are, more the chances to be a local invariant feature.

Desired properties for local features:

1. Repeatability: The same feature can be found in several images despite the geometric and photometric transformations (the place is similar despite of the light or angle.)
2. Saliency: Each feature has a distinctive descriptor.
3. Compactness and Efficiency: with just a few features you can identify an image
4. Locality: a feature is a small area of the image, but it's robust and stable.

If we cannot detect some of the same points in an image, we won't be able to find matches. We want to detect (at least some of) the same points in both images. Yet we need to be able to run the detection procedure independently per image. To achieve it, we want to extract the minimum information that allow us to put into correspondence.

I.2 Detection

Corners are distinctive image interest points. In a corner, moving in a direction will change the intensity of the pixels. Therefore, the angle of the gradient will obviously go from the darker side to the clearer one (maybe not strongly due to the module not being especially big, but that is independent of the angle). In an axis-aligned corner, we can see the Jacobian around the corner is this:

$$M = \sum_{x,y} w(x,y) \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix} = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}. \quad (1)$$

If the eigenvalues are not close to zero, the vectors are close to perpendicular in both X and Y axis. However, if they are close to zero, it will not satisfy what was mentioned above. So, we want to search for locations where both eigenvalues are large. This can be done if M is a symmetric matrix, and the decomposition is the following (where e_1, e_2 are eigenvectors and $E = \text{diag}(e_1, e_2)$ is orthogonal). As we have already told:

- The module of the eigenvalues reveals the amount of intensity change in the two principal orthogonal gradient directions.
- The eigenvectors of gives the directions of maximum and minimum change in the image.

Harris corner detector. This algorithm uses what we have learned until now:

1. Compute M matrix for each pixel of the scene, for the image window surrounding it, to get its cornerness score (how likely is the chosen window to be a corner).
2. Compute the eigenvectors and eigenvalues.
3. Find the points with larger corner response ($>$ threshold).

$$f = \frac{\lambda_1 \lambda_2}{\lambda_1 + \lambda_2}. \quad (2)$$

4. Find the local maximum in each window, i.e., perform non-maximum suppression in the neighborhood.

As we have stated earlier, the matrix decomposition gives matrix $E = \text{diag}(e_1, e_2)$ a phenomenal property: orthogonality, which directly gives invariance under rotation (i.e. $M = XEX^T$, for some orthogonal or *rotational* matrix X) to Harris Corner Algorithm, but not under scaling without some adaptations.

Select interest points in each image such that detections are scale-invariant. We can compute the derivatives with different σ to find the scale that gives the local maxima of the corneriness function f in both position and scale and scale our windows to match that σ value.

Census. It's the same intuitive idea, but it computes the derivatives with the difference of boxes. It can approximate corners and determine the optimal scale of the picture, but it is not rotation invariant.

The algorithm defines two masks, one exterior and one interior, the latter one with opposed values, and with the convolution of both masks we obtain the maximum value, which is a candidate. As the squares are a mask, it's important not to forget to normalize both masks; calculating the area of both squares as in how many pixels are in each square is enough. Interestingly, we can use any shape we want as a mask (a circle, a hexagon, an octagon...) but they must be normalized first.

This algorithm is not illumination invariant, but it is rotation and scale invariant; it's also accurate (it consistently localizes features across viewpoint change) and stable (conserves the persistency of features across the viewpoint changes).

1.3 Description of local patches

We must take into consideration that the area of the first image can be scaled, rotated and translated in the second one and be the same. We need ways to find invariances to make the comparisons easier. Rotating the patch according to its domain gradient orientation will put the patch in its canonical orientation. It can be shown that the dominant eigenvector is e_1 if $|\lambda_1| > |\lambda_2|$ and vice versa.

Photometric transformations. The problem originates because the simplest way to describe the neighborhood around an interest point is to write down the **list of intensities** from a feature vector, to convert the matrix into a list (as it is shown in the picture) is also volatile and very sensitive to small shifts and illuminations changes. As we have seen previously, the illumination changes do not affect the angle of the gradient. They just affect the module, so if we define the descriptors based on Gradient direction (SIFT).

SIFT (Scale Invariant Feature Transform). SIFT is an algorithm that uses histograms to bin pixels within sub-patches instead of just pixels according to their orientation. For each window of the image, we calculate the derivatives and make the histogram of gradient angles from the window. Finally, we concatenate all the histograms in a vector, which is invariant under photometric transformations.

- Advantages: Fast and computationally efficient. Real time and low power device. Rotation and viewpoint invariant. Significant changes in illumination.
- Applications: stereo images, recognition, motion tracking...

ORB. ORB is based on a detector that measures intensity between center pixel and those in a circular ring around the center. It's invariant under rotation, because it is based on the Intensity Weighted Center of Mass.

$$\tau(p; x, y) = \begin{cases} 1, & \text{if } p(x) \geq p(y) \\ 0 & \text{otherwise.} \end{cases} \quad f_n(p) := \sum_{1 \leq i \leq n} 2^{i-1} \tau(p; x_i, y_i), \quad n = 256b. \quad (3)$$

When we create the feature vector, we take into consideration 256 pairs of pixels around p . So, we need to find the best pair of pixels, which is the one that maximizes the variance and minimizes the correlation and also makes it illumination invariant. Unfortunately, it's not scale invariant and we should use SIFT instead. Also, it does not give us the corneriness quality measure, so we should use Harris instead.

1.4 Matching

To generate the candidate matches, we must find the most similar appearance between them (we have already done that in SSD). The simplest approach is to just compare them all and take the closest pair with a threshold. At what SSD value do we have a good match? To add robustness to the matching, we can take into consideration the ratio:

$$\frac{\text{distance to the first match}}{\text{distance to second match}} \quad (4)$$

If this ratio is low, the second match is similarly good to the first and if it's high, this implies that the distance to the second match is low, so the second could be a potential good match. A robust feature-based alignment consists of:

- Extracting features
- Compute putative matches
- Loop between: Hypothesize transformation T (small group of putative matches that are related by T). Verify transformation (search from other matches consistent with T).

Mosaic construction by Ransac. Steps:

1. Random subset of the original data (hypothetical inliers; we assume that these random points are a good representative).
2. Fit the data (hypothetical inliers) into a model.
3. Test all the other data against the model. If points fits model according a loss function is considered of the consensus set.
4. The estimated model will be reasonably good if sufficient many points have been classified as part of the consensus set.
5. We repeat the procedure a fixed number of times. If the model is rejected because too few points are part of the consensus set, we generate another. If the model is more refined than the actual, we keep it.

SIFT+Ransac és invariant al canvi d'il·luminació, també.

2 Textures

Textures can have regular patterns or random ones, but always there will be something repeated. **Applications:** *Image Classification or Segmentation, Object orientation or recognition and detect imperfections in some materials.*

Visual texture: when the uniformity of the region is perceived as a series of variations in the intensity. Causes of the visual texture appearance:

- Changes in reflectance.
- Changes in the micro-orientation of a surface.

Relevance: source of information about the scene, useful for certain visual processes (segmentation and perception of orientation).

Texture (stochastic and structural) is a property of regions:

- contextual property;
- description of the distribution of gray levels in regions more or less extensive. The texture can be perceived at different scales or levels of resolution.

A region presents a texture when the number of primitive patterns is large (not easily countable). Consider textures as made up of repeated local patterns, so:

1. Find the patterns.
 - Use filters that look like patterns (spots, bars, raw patches...).
 - Consider magnitude of response.

2. Describe their statistics within each local window.

- Mean, standard deviation.
- Histogram
- Histogram of «prototypical» feature occurrences (*textons*). Textons are defined as the visual characteristic of the image that are used in the perception of texture (extremes, crossings...).

Different textures have different attributes, so we need different filters!

Algorithm to discriminate textures. Taking in consideration that is made up of repeated local patterns, we need to find the pattern and describe their statistics within each local window. The task of finding patterns is not an easy one. The solution for this is the use of filters in the image applying convolutions, that will show us if the structure in the filter is in the image. The filter is going to be the Derivatives of Gaussians.

El que fem, doncs, és aplicar un filtre vertical (derivem sobre x) i un filtre horitzontal (derivem sobre y). Una vegada tenim les estadístiques que resumeixen els patrons en petites finestres (la mitjana i la desviació estàndard), es poden dibuixar en dues dimensions. Tenim 4 possibilitats segons on es trobin:

1. Valor petit en dimensió 1 i dimensió 2: gradient petit en ambdues direccions.
2. Valor petit en dimensió 1: finestres amb vores horitzontals, principalment.
3. Valor petit en dimensió 2: finestres amb vores verticals, principalment.
4. Valors grans en dimensió 1 i 2: gradient gran en ambdues direccions.

To know the different regions in this point cloud, we will need to calculate the Euclidean distance between the points. This is going to reveal how dissimilar is the texture from a point A to a point B. Far: dissimilar textures, close: similar textures.

Window scale. Possible to perform scale selection by looking for window scale where texture description is not changing. The solution is to have a **filter bank** of d filters. This bank will be supported by the Gaussian function, with the respective first and second derivatives. Then our feature vectors will be d -dimensional. *What filters to put in the bank?* Typically, we want a combination of scales and orientations, different types of patterns. Now we can apply all these filters and get a descriptor. Every plane is going to tell us the importance of a pixel in the plane we are analysing and how good is the filter for the image.

How to go from pixel representation to image representation? To represent the texture, we can calculate the mean of the absolute value of pixels in the image (important to be absolute, because the filters can produce negative values and black colour in the filter means negative values in Gaussian function).

$$I \rightarrow f(I) [\text{mean}_{\text{all-pixels}}(|r_1|), \dots, \text{mean}_{\text{all-pixels}}(|r_n|)] \quad (5)$$

On n és el número de filtres aplicats de tal manera que el número de plans sigui N . The problem with this technique is *not invariant to orientation or scale*, the descriptor is different.

How to recover the image with the most similar texture? We are looking for the minimal Euclidean distance (L_2) between the feature vectors ($f(I)$) of the query image and the database images.

Is the Gaussian filter-based texture recognition contrast invariant? Yes, when there is more contrast the differences are bigger (bigger is the derivative); less contrast, the derivative is smaller (Gaussian derivative).

What is the alternative to recognition of textures? **Local Binary Patterns (LBP)**. The LBP operator assigns a label to each pixel of an image by comparing each neighbor with the value of the central pixel. Binarizing the image with threshold the pixel in the centre of the mask. Similarly to ORB, LBP operator can be extended to use neighborhoods of different sizes (circular neighborhoods and bilinear interpolation of pixel values). A LBP is called uniform if and only if there are at most two transitions on the binary loop function.

Algorithm Pipe Uniform-LBP: from original image α :

1. Get a sub-image from a pixel.

2. Binarization of the image.
3. Get the value and put it in histogram.

We repeat this process for every pixel in the image and store it in the histogram. At the end we will have a LPB descriptor. For an RGB one the process is different. We pass the image to a grey-scaled one and do the algorithm. After getting the LPB descriptor we join the descriptor with the RGB histogram for each channel and get: LPB DESCRIPTOR | R HISTOGRAM | G HISTOGRAM | B HISTOGRAM.

LBP vs. Gaussian Filters.

	LPB	Gaussian filters
Pros	simple theory, fast, compact, invariant to illumination changes	lots of textures, compact, no parameters
Cons	too simple, neighbours to consider	non-invariant to illumination changes

3 Face detection

In Object Detection, the main problem is **stimulus equivalence**: there are infinite views that correspond to the same object, and we need invariant features to detect them. In some cases, we can find mechanisms associated to simple recognition of image features that are unambiguous signs of the presence of the object. But, in most cases there is no way to construct the learning complex descriptions. Differences between classify (categorize) and identify:

1. Classify or Categorize: given an object and its features, we *detect similar* objects.
2. Identify: given an object and its features, we *detect different views* of the same object.

Recognition of new objects within a class:

- We can infer the properties (uses, dangers, ...) of things we've never seen!
- Restricts the number of models for identifying (indexing);
- Allows the use of specific information to identify the class (neutralizing facial expressions);
- It enables generalization from very partial information (Mona Lisa).

Detection vs recognition: detection is to classify/find all the faces in a given image. Recognition is to identify to whom belongs the detected face.

Classification of image features goal is to define a space of image features allowing to represent objects based on their appearance (or through a set of local features) in the image.

1. Adequate representation (build descriptors of objects). Reduce the size of the data so that the invariance is kept and the other dimensions are removed.
2. Training, from a set of objects examples with their descriptors.
3. Detection or recognition of a new object instance by using its descriptor and the learned model.

Robust Real-Time Face Detection (Viola and Jones). Standard supervised classification problem. The major steps are basically three:

1. Extraction of image features.
2. Training a decision rule, called classifier.
3. Test for new images using the trained classifier.

Finding faces are always done through a sliding window across the image, so we are talking about sequential search.

3.1 Haar image features

Haar features are sequence of rescaled square shape functions that are like convolution kernels. We will apply these Haar features to all relevant parts of face to detect human faces. En altres paraules, fem masks per a cada regió característica de la cara (regió que se sol comportar de la mateixa manera).

Each feature mask, applied to the image $I(x, y)$ indicates two regions: R_1 (white) and R_2 (black).

$$F_k = \sum_{(i,j) \in R_1} I(i, j) - \sum_{(i,j) \in R_2} I(i, j). \quad (6)$$

Typically a set of K masks is used, F_k is the feature that corresponds to the k -th mask. The masks have different sizes/scales, shapes, and positions in the image regarding the square window.

Feature extraction. The feature vector describes the content of the window, and is used to train the classifier to detect the face and later, given the trained classifier to detect a face in new images.

3.2 Integral images

The integral image S is constructed as follows: the value at position (x, y) is the sum of all pixels in the image I above and left of point (x, y) . *Why are we interested in the integral images?* The Haar features are based on sums and subtractions of image rectangles and if we do all the operations point to point, the cost is huge and the detection, very slow. So, using Integral images, the cost of feature computation is reduced to computing a set of 3 sums and subtractions on the integral image points. Therefore, it's very fast.

Computing values of the I and II order Haar features at different scale and at different points of the image of the face would suppose thousands of features. *Which subset of these features should we use to determine if a window has a face?* **Re-scaling** of the image instead of generating filters of different window.

3.3 AdaBoost

AdaBoost (Adaptive Boosting) is a Machine Learning model. It is a classification algorithm that joins several weak classifiers (Decision stumps) to form a single strong classifier (strong). The construction of AdaBoost model is that every Decision Stump classifies some classes given a threshold in a feature and were misclassified in the previous classifier.

What does a weak classifier do? Want to select the single rectangle feature and threshold that best separates positive (faces) and negative (non-faces) training examples, in terms of weighted error.

Adaboost algorithm.

1. Step 1: Initialize $i = 1$.
2. Step 2: If $i == T$ (max number of iterations), go to Step 6, else go to Step 3.
3. Step 3: Given the descriptors $f(x) = (f_1, f_2, \dots, f_t, \dots)$ of a set of training objects (for which we know their class labels, face or not face), for each feature i look for the threshold θ that separates the two classes with minimum error.
4. Step 4: We choose the feature $f_t(x)$ that separates with the smallest error the training examples. The chosen classifier $h_t(x)$ is called weak classifier, t is the index of the feature.
5. Step 5: Sum the new single classifier $h_t(x)$ which corrects the error of the sum of the previous classifiers, to the previous ones. Give more priority to the wrongly classified samples (faces and no faces).

$$F_t(x) = F_{t-1}(x) + h_t(x), \quad t = 1, \dots, T. \quad (7)$$

6. Step 6: The ultimate classifier is the sum of the weak classifiers added at all iterations.

3.4 Classifiers cascade

To improve the result, the algorithm of Viola-Jones doesn't want to lose any false negative, this means they want to recognize all the faces. They don't care much if there is a false positive.

Cascade of classifiers, algorithm.

1. Each classifier is an AdaBoost classifier.
2. The first classifier C_1 is the simplest of all, and only classifies based on two features.
3. The following classifiers are more complex and use more features to refine the results of previous classifiers.
4. Each subsequent classifier C_i is trained with the error of the classifier C_{i-1} .
5. For each classifier C_i (during the training phase), we decide the FP_r value we want to obtain, and add features until the decided value is not achieved! (N_f is the number of features).
6. If FP_r of C_i is not enough, we add another classifier, C_{i+1} .

Each classifier in the cascade reduces the number of false negatives, keeping the number of successes (TP). The cascade of classifiers enables a very low rate of false negatives by detecting faces in real time.

4 Face recognition

For face recognition we must do facial analysis due to socio-cultural reasons (emotion recognition) and evolutionary reasons (identification). It can give us plenty of information: attention focus, age, gender, etc.

4.1 The problem

Face analysis the predecessor of biometrics systems, which are automated systems of verifying or recognizing the identity of a person based on some physiological characteristics (fingerprints, iris or face pattern) or behavioral aspects (handwriting). They are one of the first problems of automated facial analysis. General problems and difficulties:

1. Spatial resolution degrades the automatic recognition systems.
2. Facial expressions, pose and illumination affect facial recognition.
3. Equivalence between stimuli. In some cases the object recognition can find simple mechanisms of recognition associated with simple characteristics of images that are unambiguous signs of the presence of the object.
4. Dimensionality of the problem and Data redundancy. *Solution:* Instead of storing features or 3D models, directly store a collection of many views of the object (difficult to store all).
5. Face images are extremely high-dimensional, and few vectors correspond to valid faces images. So, we want to effectively model the subspace of face images

Face recognition: To define a space of image features that allow to represent objects based on their appearance (or a set of local features) in the image.

4.2 Eigenfaces

We want to construct a low-dimensional linear subspace that best explains the variation in the set of face images. In the feature space, a pixel corresponds to an axis. So, if one pixel has no value (doesn't belong to a face), we need to discard it. To optimize this procedure, we want to construct a low-dimensional linear feature subspace that best explains the variation in the set of face images. In other words, that allows us to find all the combinations of faces.

Each image has m rows and n columns so it defines a vector in $\mathbb{R}^{n \cdot m}$. Steps:

1. Find features or eigenfaces $(x_1, \dots, x_{m \cdot n})$: representation of faces as points in high dimensional space.
2. Train the classifier KNN in the $\mathbb{R}^{n \cdot m}$ space: it assigns the majority label of the k closest neighbors of the training set.

We look for a transformation of the original space to a smaller space ($M \ll mn$), where faces are represented with their coordinates in this new space $\mathbb{R}^M$.

Dimensionality and redundancy: Dimensional reduction of the original data.

$$T : (x_1, \dots, x_n) \longrightarrow (y_1, \dots, y_m), \quad m \ll n. \quad (8)$$

Removing and minimizing non-relevant information and retaining information necessary to classify, recognize, etc. If you want to classify a face X_i , we need to project it into the new reduced space and apply the KNN classifier:

$$Y_i = (X_i - \bar{X})^T \cdot (u_1, \dots, u_M) \longrightarrow (w_1, \dots, w_n). \quad (9)$$

Here we are applying a change of base: $(u_1, \dots, u_M)$ is the canonical basis of the reduced $\mathbb{R}^M$ space and we are multiplying it with the face point we want to transform. All in all, we are removing the redundant information and projecting $\mathbb{R}^{nm}$ space into the reduced space $\mathbb{R}^M$. and applying the classifier knn considering the k projected neighbor faces of the training set.

The data are not always uniformly distributed. The technique that allows to find the reduced subspace where data live, is called: Principal Component Analysis (PCA). Compute:

1. Eigenvectors $(e_1, \dots, e_M)$ of the covariance matrix, which define the axis of maximum variance.
2. Eigenvalues $(\lambda_1, \dots, \lambda_M)$, which give a measure of the variance of the data. The larger the λ_i , the more variance in the data vector in the direction of eigenvector u_i . We are only interested in the first k eigenvectors of the largest eigenvalues.

How to find the best sub-space to represent the face family?

1. Given M images of size mn , we'll construct their vector $X_i = (x_1, \dots, x_M)$ in the original space $\mathbb{R}^{mn}$.
2. Then, we'll compute the mean image $\bar{X} = \frac{1}{M} \sum_i X_i$.
3. Covariance matrix is Σ such that:

$$\Sigma = \frac{1}{M} \sum_{i=1}^M (X_i - \bar{X})(X_i - \bar{X})^T = AA^T. \quad (10)$$

4. Get its eigenvalues and eigenvectors. The eigenvectors of the covariance matrix are called eigenfaces. Eigenvectors $(u_1, u_2, \dots)$ of the covariance matrix Σ define λ_2 the subspace that represents the data (faces) distribution.
5. The original image once centered is projected onto the eigenspace: $Y_i = (X_i - \bar{X})^T \cdot (u_1, \dots, u_M) \longrightarrow (w_1, \dots, w_n)$.

Properties of PCA:

1. Seeks directions efficient for representing data in all its variance.
2. Reduces the dimensions of data.
3. Accelerates the execution time of the algorithms.
4. Reduces noise.

Disadvantages of PCA:

1. Linear decomposition
2. Orthogonal axes
3. Represents a transformation consisting of a rotation, affine rescaling and tran
4. Is not discriminative.

Recognition with eigenfaces:

1. Training: Given the training set, we compute the eigenfaces $U = (u_1, \dots, u_m)$ of AA^T .
2. Recognition: Given a new face, we center it and project it into the reduced space.
3. Classify: by a KNN classifier in $\mathbb{R}^M$.

Finding eigenvectors and eigenvalues of AA^T is an untreatable problem, we can consider matrix $A^T A$ instead, which has size $M \times M$ (much smaller) because they have the same eigenvalues.

Final algorithm to obtain the eigenfaces.

1. Construct the matrix $A^T A$ of size $M \times M$ where the columns are centered faces: $A = [X_1 - \bar{X}, \dots, X^M - \bar{X}]$.

2. Compute the eigenvectors $V = [v_1, v_2, \dots, v_m]$ matrix of $A^T A$.
3. Sort by magnitude of their corresponding eigenvalues and keep the most important eigenvectors (with higher eigenvalues).
4. The M eigenfaces are obtained by multiplying the matrix A with v_i :

$$u_l = \sum_{k=1}^M v_{lk} A_k, \quad l = 1, \dots, M, \quad (\text{II})$$

where A_k are the columns of matrix A .

Since eigenfaces are forming a base, we can express a face as a linear combination of eigenfaces. The method of eigenfaces recommends that the faces are aligned, usually eye-centered. The representation of the original faces X in the reduced space would be the vector:

$$\vec{Y}_i = (X_i - \bar{X})^T \cdot (u_1, \dots, u_k) = (w_1, \dots, w_k) \implies X_i = \bar{X} + \sum_{i=1}^k w_i u_i. \quad (\text{I2})$$

4.3 Eigenfaces for face detection

The space defined by the eigenfaces can be viewed as a subspace of the space of images. We can always calculate the distance between any image and the face subspace! And use it as a criterion for deciding whether an image is a face or not.

To detect a single face:

1. First, we have to project it in the reduced space:

$$VecY_j = (X_j - \bar{X})^T \cdot (u_1, \dots, u_k). \quad (\text{I3})$$

2. Then, we'll reproject it in the original space:

$$\hat{X}_j = \bar{X} + \sum_{i=1}^k w_i u_i. \quad (\text{I4})$$

3. We obtain the difference $X_j - \hat{X}_j$.

Global appearance method: not robust to misalignment, background.

It has the limitation that it works only when the faces are aligned and have the same scale and pose angle.